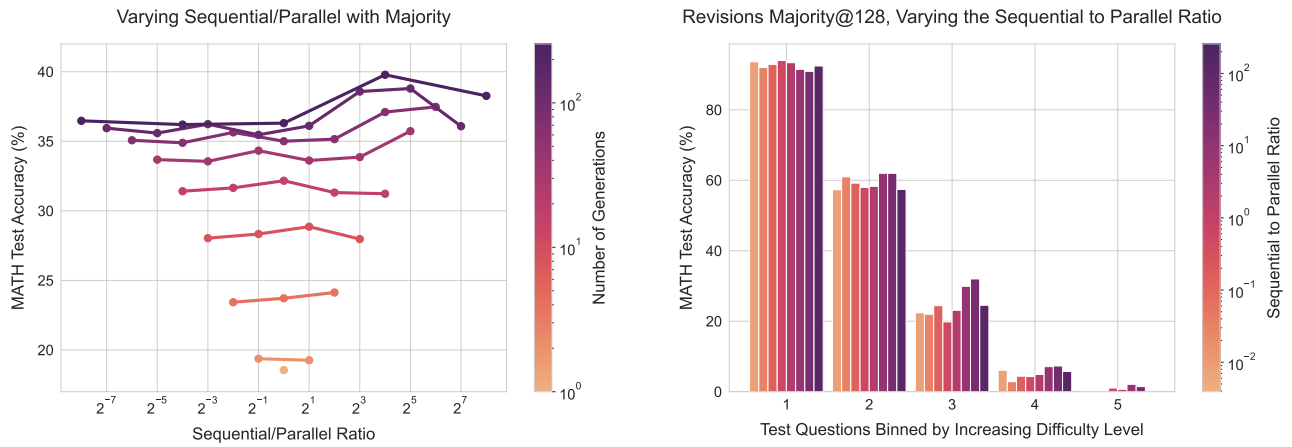

Figure 10 | Varying the ratio of generation budget allocated to sequential verses parallel samples, using majority to select the answer, rather than the verifier. **Left:** Each line represents a fixed generation budget as the ratio is changed. We see that similar to the verifier case, in the majority case, there exists an ideal ratio of sequential to parallel test-time compute at a given budget. **Right:** Analyzing performance across difficulty bins, we see that the easier questions are mostly invariant the ratio of sequential to parallel, whereas on the harder questions there is an ideal ratio of sequential to parallel test-time compute.

the question. We then bin the value for each question in the test-set into five quintiles (using the same procedure as the oracle difficulty bins). We refer to this as "predicted difficulty" rather than "oracle difficulty". Technically this procedure is extremely costly because it requires generating many samples. While we do not account for this cost in our analysis, in a practical production setting, this cost would be problematic. A more efficient approach would be to finetune a model to predict correctness directly, given the question. We do not explore this in our work, but leave such exploration of cheaper methods of estimating difficulty to future work.

In Figure 12 we plot PRM-search results using our difficulty bins, and in Figure 11 we plot the corresponding revision results. We see that in both settings these predicted bins demonstrate similar trends to the oracle bins.

D. PRM Training Details

We finetune our PRM as a binary classifier, where the model predicts a value between 0 and 1 at each step in the solution. We train the model with soft values obtained from the monte-carlo rollouts, using a binary cross entropy loss function (e.g. $-(ylog(\hat{y}) + (1-y)log(1-\hat{y}))$ where y corresponds to the soft ground-truth value and $\hat{y}$ the model's predicted value). We finetune the model base model using the AdamW optimizer, with lr 3e-5, batch size 128, dropout 0.05, and Adam betas $(0.9, 0.95)$. We conduct early stopping, selecting the checkpoint with the lowest validation loss on a random held-out validation set, consisting of 10% of the questions in the original PRM800k training split.

We finetune the PRM on 16 samples per question from the corresponding few-shot prompted base model. At each step, we use 16 monte-carlo rollouts, using the same base model and prompt, to estimate the step-level value. We filter out all samples which fail to output a valid, parsable final answer from the training data, as we found these to hurt PRM performance in initial experiments.

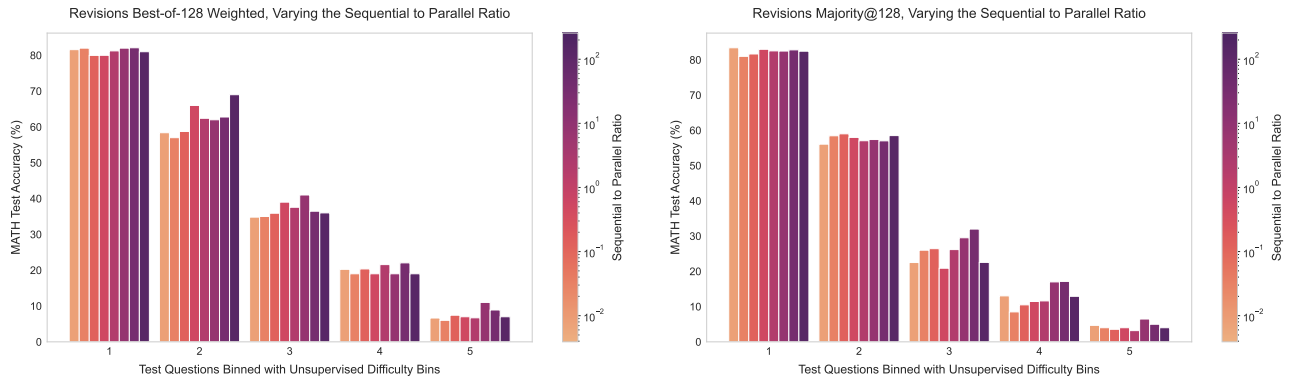

Figure 11 | Using our PaLM 2-S* PRM to compute difficulty bins without ground truth correctness information for revisions. On the left we plot verifier selection and on the right we plot majority selectionl We see largely similar performance trends with these bins as we do with the ground truth ones in Figures 7 and 10.

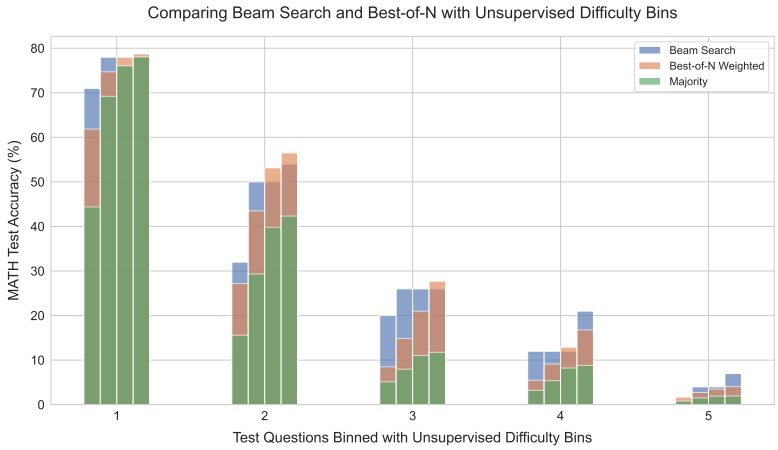

Figure 12 | Using our PaLM 2-S* PRM to compute difficulty bins without ground truth correctness information for PRM search. We see largely similar performance trends with these bins as we do with the ground truth ones in Figure 3.

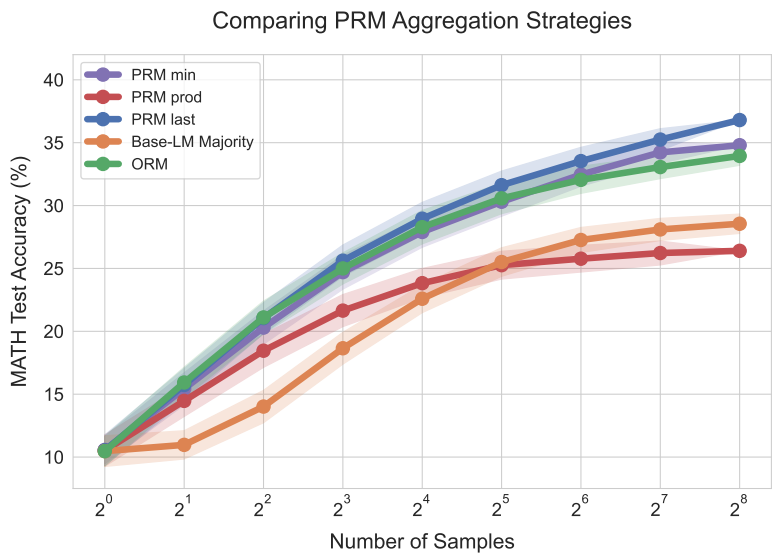

Figure 13 | We compare different methods of aggregating per-step PRM scores to produce a final score for the full solution: "min" refers to taking the minimum score accross all steps, "prod" takes the product of all step correctness probabilities, and "last" just uses the last step score. We see that last performs the best across all aggregation strategies.

When generating the samples, the base model is prompted to output answers in newline separated a step-by-step format, as done in Lightman et al. [22]. We then separate each of the answers into steps using a simple newline splitting procedure. We include details about our prompt in Appendix G.

E. Comparing PRM Aggregation Strategies

We compare different methods of aggregating per-step PRM scores to produce a final score for the full solution. Specifically we compare: 1) taking the minimum score accross all steps as done in Lightman et al. [22] (e.g. "min"); 2) taking the product of all step correctness probabilities (e.g. "prod"); and 3) taking just the last step prediction (e.g. "last"). We see in Figure 13 that taking the last step outperforms the other two approaches. Prior works [22, 45] found min to be the best aggregator. We believe that the discrepancy is due to the fact that our verifier was trained with soft MC return labels, which surface very differently from binary correctness labels, and therefore other aggregation strategies may not have the same effect.

Interestingly, when using the last step aggregation, we are effectively using the PRM like an ORM. However, we see that the PRM outperforms the ORM, suggesting that in our case the per-step PRM training may be largely useful as a form of representation learning, rather than purely as a tool at inference time. Future work should further explore this line of reasoning.

F. Comparing PRM and ORM

We trained a PRM and ORM model using the PaLM 2-S* base LM. We see in Figure 14, that the PRM outperforms the ORM, and the gap between the gap between the PRM and ORM grows with the number